CeTTa Project Roadmap:
A Territory Map for MeTTa Runtime Design

Zar Goertzel Oruži

Living document, started 2026-04-08

Abstract

This document describes the *territory* of programming language design decisions relevant to CeTTa, a direct C runtime for MeTTa. It is not a checklist of steps to complete, but a map of the landscape: regions to explore, peaks worth climbing, valleys to avoid, and clouds of possibility where the best path remains uncertain. We begin with an idealized vision of what CeTTa could become at its very best, then survey the terrain of term representation, evaluation semantics, search control, and storage. The journey so far is described as a continuous narrative, and future directions are presented as open questions rather than prescribed stages.

Contents

Part I
The Ideal

1 What Is the Very Best CeTTa Can Be?

Before examining techniques, trade-offs, or current status, we ask the fundamental question: if we could design CeTTa perfectly, what would it look like? This is the *Way of the Dotts*—define the idealized endpoint first, then work backwards. The destination shapes the journey.

1.1 The Vision

At its best, CeTTa would be a **semantically authoritative, high-performance MeTTa runtime** with these properties:

HE Fidelity as Baseline. The Hyperon Experimental (HE) semantics is not an aspiration but the foundation. Every well-typed MeTTa program that runs correctly in HE should produce the same observable results in CeTTa. Where CeTTa extends beyond HE (and it may), those extensions are explicit and documented, never silent behavioral drift.

Three Cleanly Separated Concerns. The mature systems we admire—E Prover, XSB, SWI-Prolog, Vampire—all eventually separate:

- **Semantics**: what the language means, independent of how results are computed.
- **Search and control**: how the runtime explores the space of possible reductions, manages nondeterminism, and returns results.
- **Storage and indexing**: where terms live, how they are retrieved, and what backends support them.

These concerns interact, but they are not the same thing. A clean separation means you can improve indexing without touching semantics, or change search strategy without breaking storage invariants.

One Shared Term Universe. Every persistent term should have a stable identity owned by one canonical store. Spaces are views or membership layers over that universe, not separate physical heaps. This is what Schulz discovered after years of "a mess" with multiple mutable term banks in E Prover: the converged design uses one immutable term bank with multiple specialized indices.

Specialized Engines as Extensions, Not Core. PLN backward chaining, ATP integration, factor-graph demand propagation, MORK algebraic operations—these are valuable, but they are not the evaluator core. They plug in through explicit interfaces. The evaluator itself is one coherent substrate, not a patchwork of special cases.

Performance Competitive with PeTTa. On recursive search benchmarks that exercise the WAM's strengths, CeTTa should be competitive with PeTTa while preserving HE semantics that PeTTa diverges from. This is an engineering challenge, not a compromise of correctness.

Formal Verification as Confidence Anchor. Key invariants should be backed by Lean theorems: term canonicity, evaluator path equivalence, binding safety. Not as checkboxes to tick, but as anchors that give us confidence when we're uncertain.

1.2 One Semantic Contract, Many Execution Lanes

The best possible CeTTa is not "a faster C port of HE." It is a **family of semantically equivalent execution engines** living under one contract. At minimum, we envision:

Reference evaluator: Small, clear, easy to compare with Meta-MeTTa / HE ideas. Used for semantic reference and differential testing.

Production evaluator: Preserves direct tail reentry for singleton deterministic positions; uses resumable frames only where semantically necessary; supports streaming, bounded demand, and multiple results.

Tabled evaluator: Adds SLG-style suspension/resumption for recursive and cyclic search. Can grow from revision-aware caching toward full variant/subsumptive tabling.

Compiled/derived machines: Bytecode interpreter, WAM-like compiled control, or generated specialized evaluators for hot paths. Highest performance ceiling, greatest implementation complexity.

The core obligation: **all lanes must agree on observable results where they overlap**. This is what makes it one system, not a patchwork.

1.3 What We Don't Know

The vision above describes properties, not implementations. We have beliefs about how to achieve them, but we acknowledge uncertainty:

- **Eval-local term sharing**: Borrowing canonical identity into evaluation arenas could reduce memory pressure significantly. Or it could introduce lifetime complexity that costs more than it saves. The Conchon–Filliâtre weak-reference model [1] suggests a path, but we haven't walked it yet.

- **The right tabling granularity**: Full SLG suspension/resumption/completion (XSB-style) is powerful but complex. Simpler revision-aware memoization may suffice for many workloads. We don't know where the sweet spot lies.
- **Contextual normal-form**: The question "is this term a value?" depends on expected type, evaluation context, and space contents. The literature on dependent types (Martin-Löf, Coquand) treats this carefully; we're still learning how it applies to MeTTa's specific type discipline.
- **Demand propagation**: When a bounded consumer (like `select`) needs only one result, the producer should stop. How to thread this demand through nondeterministic evaluation is an open research question.

The roadmap that follows describes the *territory* where these questions live, not definitive answers.

1.4 What CeTTa Users Can Do

The vision above is architectural. But a runtime exists for users, not architects. At its best, CeTTa lets you:

- **Load very large knowledge bases**—millions of atoms across distributed backends (DAS-scale)—without reimplementing your application.
- **Run deep recursive backward chaining** without stack failure. Recursive PLN queries over large knowledge graphs should complete, not crash.
- **Get bounded-demand answers** without overproduction. If you need only the first result, the evaluator should not compute thousands.
- **Switch backends without changing semantics**. Move from in-memory testing to PathMap persistence to distributed MORK without altering your MeTTa code or expecting different answers.
- **Stream aggregations** without full materialization. Fold over results as they arrive, not after collecting them all.
- **Trust theorem-backed invariants** on critical seams. When a Lean theorem says binding projection is sound, you can rely on it.
- **Run concurrent queries** over immutable snapshots. Parallel exploration of a frozen space state should be safe and deterministic.

These are user powers, not implementation checkboxes. The endgame is not "we have tabling" but "you can query recursive knowledge without crashing."

Part II
The Landscape

2 Term Representation and Identity

Every operation in a term-rewriting runtime—pattern matching, binding application, result collection, memoization—operates on terms. The representation of terms therefore determines the cost of everything built on top. This is not speculation; it is the consensus of Knuth, Ritchie, Warren, Schulz, and every systems researcher who has built a mature symbolic runtime.

2.1 Why Representation Comes First

If terms are not shared, every `bindings_clone`, every pattern match, every result-set entry pays a representation tax that no local optimization can eliminate. You can tune individual operations, but you cannot escape the fundamental cost of copying.

Schulz's E Prover provides the clearest case study. It began with mutable shared terms and multiple term banks. This was, in Schulz's words at IWIL 2025, "a mess." The mature E design converged on *one immutable term bank* with garbage collection, cached rewriting, and multiple specialized retrieval indices. The lesson: the architecture you resist at the start is the one you'll eventually need.

Zhu et al. (JuliaSymbolics, 2025) quantified this. They bolted hash-consing onto an existing dynamically-typed symbolic runtime and measured 3.2× speedup and 2× memory reduction on a real biological signaling model (1,122 ODEs, 24,388 reactions). XSB's manual states it directly: "interned ground terms are stored once in a global area; when tables are declared `intern`, those ground terms need not be copied into and out of tables."

2.2 Hash-Consing: The Foundation

Hash-consing is the technique of storing structurally identical terms exactly once, returning the same pointer for equal structures. The benefits compound: equality becomes pointer comparison, copying becomes aliasing, and memory usage becomes proportional to structural diversity rather than usage count.

The canonical reference is Conchon and Filliâtre's 2006 paper on type-safe, modular hash-consing in OCaml. Their key insight: strong references in a hash table prevent collection and create dangling pointers when referents die. The solution is weak-reference hash tables where the garbage collector sweeps dead entries.

For a C runtime without GC, the question becomes: how do we get the benefits of hash-consing without the lifetime bugs? Two approaches appear in the literature:

Persistent-only hash-consing. Terms in the persistent arena are hash-consed; evaluation-local terms are not. Simple, but pays the copy tax during evaluation.

Arena-disciplined hash-consing. Evaluation arenas can borrow canonical terms from the persistent universe. Terms created during evaluation are local to that arena; when the arena resets, they vanish. Terms that need to persist are "promoted" to the persistent arena (and hash-consed there) before the evaluation arena dies.

The second approach is more powerful but requires careful lifetime discipline. An `ArenaResetSafety` theorem—formalizing that published terms survive arena rewind—would give confidence before attempting it.

2.3 SymbolId: A Representational Cutover

Profiling in March 2026 showed ∼40% of CPU time in symbol string operations: interning, checking atom type, comparing strings. The fix was not a local optimization but a *representational cutover*: `SymbolId` (`uint32_t`) for global spellings, strings only at I/O boundaries.

This mirrors Rust's own `Symbol` (interned index) and first-order prover functor-code disciplines (Vampire, E). The result: 38% instruction reduction on typical workloads.

2.4 The One-Universe Question

The ideal described in Part I calls for one canonical term universe with spaces as views over it. Currently, `Space` still owns `Atom **atoms`, and the `TermUniverse/term_canon` seam is a storage-policy shim rather than the one true store.

An important nuance from PathMap investigation: because PathMap is a prefix-sharing trie, naively encoding space membership as a leading prefix can *reduce* cross-space sharing (two otherwise identical terms diverge at the first path component). Canonical term storage should be separated from space membership indexing.

2.5 Clouds of Possibility

Persistent-only vs. eval-local sharing: The conservative path hash-conses only persistent terms; the aggressive path borrows canonical identity into evaluation. We don't know which is better for CeTTa's workloads. Positive witnesses would include: backward chaining benchmarks completing where they currently OOM, hashcons-hit counters rising, and memory usage approaching PeTTa's levels on comparable workloads.

Explicit substitutions: Krishnaswami (citing Abadi et al.) proposes deferring `bindings_apply` by representing `(term, substitution)` closures instead of eagerly copying. This eliminates the most expensive per-call operation but requires the evaluator to handle closures throughout. A long-term direction, not a near-term change.

2.6 Reflective Evaluation

MeTTa can quote and evaluate arbitrary expressions, including expressions that describe its own evaluator. This creates what Smith [15] called a *reflective tower*: a conceptually infinite stack of meta-circular interpreters, each interpreted by the one above.

Amin and Rompf [16] showed that such towers can be *collapsed*—stage polymorphism allows interpreters to compose in a way that eliminates interpretive overhead. Their Pink and Purple languages demonstrate that arbitrarily deep self-interpretation can be optimized away.

For CeTTa, reflection raises practical questions:

- **Tabling and self-modification**: If a tabled query's definition changes mid-evaluation (via `add-atom`), what happens to cached answers? This is not hypothetical—PLN rule learning can modify the space while reasoning proceeds.

- **Meta-circular invariants**: When MeTTa evaluates a MeTTa evaluator, which invariants (typing, binding safety) transfer to the inner evaluation?
- **Collapsing for performance**: Can we detect when nested `eval!` calls are semantically equivalent to direct evaluation and optimize accordingly?

We do not have answers. But ignoring reflection would be ignoring one of MeTTa's distinctive features. The reflective tower is not a bug or an edge case; it is the language's design center for AGI self-improvement scenarios.

3 Evaluation Semantics

The evaluator is the heart of the runtime. It takes an atom (or outcome) and produces a stream of reduced atoms. Everything else—REPL, file loading, FFI—feeds into or out of the evaluator.

3.1 HE's Principal Evaluation Stages

The HE specification in `metta.md` describes evaluation through mutually recursive functions. The principal stages are:

`metta`: Entry point; handles special forms, dispatches to interpretation.

`interpret_expression`: Inspects types, checks function-type applicability, dispatches.

`interpret_function`: Typed head/argument evaluation.

`interpret_args`: Evaluates arguments according to their declared strictness.

`interpret_tuple`: Handles tuple construction.

`metta_call`: Grounded execution and equation lookup.

These are the actual HE names. Prose labels like "EvalAtom" or "InterpretExpression" are fine for discussion, but the spec names matter when checking fidelity.

The key insight: `if` should not be a hardcoded C handler. The function type `(: if (-> Bool Atom Atom $t))` declares that the condition (`Bool`) is strict and the branches (`Atom`) are lazy. `interpret_args` reads the type and evaluates accordingly. The stdlib equation `(= (if True $t $e) $t)` then works through the general path. Per-op handlers are a tempting shortcut that becomes architectural debt.

3.2 The Contextual Nature of "Value"

When is a term fully reduced? The naive answer—"when no equations match"—is incomplete. In HE, the answer depends on:

- The **expected type**: an expression may be returned unchanged if the expected type is `Atom`.
- **Evaluated markers**: an expression already marked evaluated is not re-evaluated.
- **Atom category**: variables, errors, and empty are immediately returned.
- **Grounded dispatch**: a grounded function may apply even though no equations match.

- **Space contents**: the same term may reduce in one space and not another.

The dependent types literature (Martin-Löf, Coquand, Pfenning) treats normalization carefully because type checking depends on it. The principled answer comes from *normalization by evaluation* (NbE): evaluate into a semantic domain, then read back to syntax [17]. MeTTa's type discipline is softer, but the same contextual structure applies. A bare predicate `IsValue(t)` is probably the wrong abstraction; something like `NormalFormAt(space, expectedType, atom, bindings)` captures the dependencies more faithfully.

3.3 TCO: A Constraint, Not a Checkbox

Tail-call optimization for deterministic single-result forms (`let`, `chain`, `let*`, `if`, `case`, `function`/`return`) is not an optional performance feature. It is a semantic constraint: deeply recursive MeTTa programs must not blow the C stack.

The `TAIL_REENTER` macro in CeTTa's `eval.c` implements this as a `goto tail_call` trampoline. It is cheap, correct, and *must be preserved* in any refactoring. The core-opt epoch's central mistake was replacing `TAIL_REENTER` for `let`/`chain` with full frame allocation—a semantic regression that produced wrong answers, not just slowdowns.

3.4 Unification and Cyclic Terms

What counts as a valid binding? Logic runtimes differ materially here. SWI-Prolog supports rational trees (cyclic structures) and exposes `unify_with_occurs_check/2` for those who want to reject cycles. XSB takes a different approach. This is not a performance detail—it changes what programs mean.

For CeTTa, this remains an **open design question**:

- Does CeTTa reject cyclic bindings by default? HE appears to reject them (no rational tree support), but whether this is fundamental or could be relaxed remains open.
- Should rational trees ever be supported as an opt-in mode?
- Is occurs-check always semantic, or configurable per-lane?
- How do compiled/tabled lanes preserve unification policy consistently?

The answer affects what matchers must reject, what tables can store safely, and what alpha-equivalence means. Settling this is prerequisite to serious tabling and compiled-lane work.

3.5 Clouds of Possibility

Frame-based evaluation for typed-apply: Typed ordinary application with multi-step argument evaluation benefits from heap-allocated frames. The question is: can we isolate this to typed-apply only, preserving `TAIL_REENTER` for everything else?

Direct vs. stack equivalence: If we have both recursive and frame-based evaluation paths, they must produce identical results on the overlapping subset. A Lean theorem (`DirectVsStackEquivalence`) would anchor this.

Demand propagation: When `select` or `once` bounds the consumer, the producer should stop producing. This requires either generator-style suspension or explicit demand threading. Neither is fully designed.

4 Search and Control

MeTTa is nondeterministic: a single expression can reduce to multiple outcomes. Search and control determine how the runtime explores this space and returns results.

4.1 WAM Foundations

The Warren Abstract Machine (WAM) remains the gold standard for efficient Prolog execution after four decades. Its core ideas:

Choicepoints: Snapshots of state before nondeterministic choices.

Trail: A log of bindings made since the last choicepoint.

Backtracking: Undo the trail, restore the choicepoint, try the next alternative.

Last-call optimization: If no choicepoint remains in the current frame, reuse the frame for the tail call.

These ideas transfer to any nondeterministic language, though MeTTa's type-directed evaluation adds complexity the WAM doesn't face.

A crucial WAM distinction: **environments** (AND-nodes) are call frames holding local variables and return addresses; **choicepoints** (OR-nodes) are snapshots taken before nondeterministic choices [3]. Environments grow and shrink with deterministic calls; choicepoints freeze the stack until backtracking. In CeTTa, `SearchContext`/`ChoicePoint` map to OR-nodes, while `NativeResumeStack`/`EvalFrame` serve as AND-nodes for streaming evaluation. Keeping this distinction clear prevents frame-management bugs that plagued early implementations.

4.2 Tabling: The Ladder of Capabilities

Tabling (memoization of goals and their answers) eliminates redundant computation and handles left-recursive predicates that would loop in naive evaluation. XSB is the reference implementation; SWI-Prolog provides a modern alternative with delimited continuations. Scryer-Prolog implements tabling via the Desouter et al. approach [6].

The tabling landscape forms a **ladder of capabilities**:

Level 1: Revision-aware caching. Store query results keyed by `(space, revision, pattern)`. Invalidate when the space changes. Simple, but coarse-grained—any change invalidates all entries for that space.

Level 2: Variant tabling. Cache based on structural equivalence of query patterns (ignoring variable names). `foo(X, Y)` and `foo(A, B)` share results. More reuse than simple memoization.

Level 3: Suspension/resumption/completion. Consumers block on incomplete tables; producers resume when new answers arrive. When all producers finish, tables are marked complete. This is full SLG resolution—required for recursive queries that don't terminate under naive evaluation.

Level 4: Subsumptive tabling. Reuse answers from more general queries for more specific instances. If `foo(X)` is cached, `foo(bar)` can reuse those results filtered by unification. Greater sharing, more complex implementation.

Level 5: Answer subsumption. Remove redundant cached answers when one subsumes another. Important for lattice-valued or aggregating queries (shortest path, best proof).

Level 6: Incremental tabling. Track dependencies between facts and cached queries. When a fact changes, invalidate only transitively-dependent entries. Essential for applications with frequent updates (dynamic knowledge graphs, belief refinement).

Level 7: Monotonic/shared/parallel tabling. Monotonic tabling restricts queries to return monotonically increasing sets (terminates earlier). Shared tables across threads enable parallel search. These are advanced features in mature systems.

CeTTa need not climb the full ladder immediately. For many workloads, Level 1–2 suffice. But for recursive PLN chaining over large knowledge bases, Level 3+ becomes essential. The question is: which workloads justify which levels?

4.3 Current Seams

CeTTa's `SearchContext` and `ChoicePoint` structures in `search_machine.h` are real, small abstractions. They're not full SLG, but they're not nothing either. `TableStore` provides revision-aware memoization infrastructure.

The `OrderedOutcomeVisitor` and ordered-visitation seam handle streaming results in the current evaluator. This is the path where `collapse`, `fold`, and `select` operate.

4.4 Search Fairness and Completeness

Depth-first search is simple and memory-efficient, but incomplete: it can loop forever on infinite branches, never reaching solutions on other paths. Breadth-first search is complete—it finds every solution—but uses memory exponential in depth. This is not an abstract concern; recursive PLN queries can easily produce infinite search spaces.

The literature offers several solutions:

Iterative deepening [18]: Run depth-limited DFS with increasing limits. Achieves BFS completeness with DFS memory. The overhead of re-expansion is $\sim b/(b-1)$ where b is branching factor—acceptable for most workloads.

Interleaving (miniKanren) [19]: Interleave exploration of disjuncts, guaranteeing no branch starves indefinitely. Complete even with infinite branches, as long as some delay occurs in recursive cycles.

Bounded-depth with tabling: Use cycle detection in the table to prevent infinite loops. SLG resolution (Level 3) handles this naturally.

CeTTa currently uses essentially depth-first evaluation with some ordered-streaming structure. This is not a bug but a known limitation. The tabling ladder provides the path to completeness for recursive queries. Whether to adopt interleaving (which changes observable result order) or rely entirely on tabling (which adds infrastructure) remains an open design question.

4.5 Parallel and Concurrent Execution

The scale story matters. A runtime that can only use one core on a 128-core machine is leaving performance on the table. Options for parallelism:

Snapshot-parallel query: Freeze the space as an immutable snapshot, then run multiple query threads with thread-local scratch. No locks needed for reads; writes are forbidden. Simple and effective for read-heavy workloads.

Shared tabling: XSB-style thread-shared tables enable parallel producers feeding shared consumers. Coordination overhead exists but can be amortized over large result sets.

Rho-calculus concurrency: Meredith's reflective higher-order process calculus [20] provides formal process-algebraic semantics for concurrent rewriting. MeTTa's quote/unquote naturally maps to rho's `@` and `*` operators, offering a principled foundation for parallel execution without ad-hoc concurrency semantics.

DAS distribution: The Distributed Atomspace [9] shards knowledge across machines. For million-atom scales, this is not optional but necessary.

A fundamental question: is MeTTa evaluation deterministic? If so, parallel execution is semantic-preserving and result order can be relaxed. If not (as with `random` or concurrent space mutation), the semantics of parallel evaluation must be carefully defined.

4.6 Bounded Demand and Streaming

Many queries need only partial results:

- `select k` — return first k answers, stop producing
- `once` — return first answer only
- `fold` — aggregate results incrementally without full materialization
- Streaming consumers — process answers as they arrive

These are not mere conveniences. For large nondeterministic searches, bounded demand is the difference between "finishes in milliseconds" and "runs forever computing unused results."

The challenge: threading demand through an eager evaluator. Options include generator-style suspension (yield and resume), explicit demand tokens propagated through choicepoints, or pengines-style incremental retrieval. None is fully designed for CeTTa.

4.7 Clouds of Possibility

Full SLG vs. simpler caching: Implementing full suspension/resumption/completion is substantial work. For which workloads would it matter? Recursive PLN chaining? Large knowledge-base queries? We should identify concrete witnesses before committing.

Demand termination: Bounded consumers should stop unbounded producers. This is easy to state, hard to implement in an eager evaluator. Lazy streaming or explicit demand tokens are two approaches.

Parallel search: Snapshot-parallel query over immutable space snapshots with thread-local scratch enables parallelism without locks. But MeTTa's semantics currently assume deterministic ordering in some places. Relaxing this requires semantic clarity first.

4.8 Result Ordering

Result ordering depends on the search strategy and engine selected. CeTTa does not promise a fixed presentation order by default—only multiset equality of results. A deterministic-order mode could be offered as an explicit option for reproducibility-sensitive workloads.

The HE spec is authoritative here; CeTTa follows whatever ordering guarantees HE makes (or doesn't make). If HE specifies deterministic order for certain operations, CeTTa must match. If HE leaves order unspecified, CeTTa may relax it for parallelism or other optimizations.

5 Storage and Indexing

Where do terms live? How are they retrieved? These questions intersect with term representation but are not identical.

5.1 Discrimination Trees and Substitution Trees

First-order theorem provers index their clause sets for fast retrieval. Two structures dominate:

Discrimination trees (E Prover). Index by functor, then by argument structure. Fast for forward reasoning and rewriting. Good when you want all terms matching a pattern.

Substitution trees (Graf 1994). Combine discrimination-tree speed with abstraction-tree generality. Crucially, they produce bindings during traversal, not just matching terms. This is exactly what pattern matching in MeTTa needs.

CeTTa's `subst_tree.c` explicitly says it produces partial bindings during the walk—Graf-style.

5.2 One Universe, Many Spaces

The ideal: one canonical term universe, with spaces as membership layers indexed separately. The current reality: `Space` owns its own `Atom **atoms`, and migration hasn't begun.

PathMap offers an alternative storage backend: a prefix-compressed trie where common prefixes share nodes. It supports algebraic operations (union, intersection, difference) that native arrays don't.

The Distributed Atomspace (DAS) [9] represents another backend direction: very large knowledge bases across machines, with a MeTTa-integrated query API and multiple persistence engines. For million-atom scales, DAS-style distributed backends become essential.

The question for all backends is **semantic parity**: given the same logical space contents, do native, PathMap, MORK, and DAS backends produce identical evaluation results? This is a formal obligation, not just a testing goal.

5.3 Revision-Aware Caching

When evidence is added or removed, which cached results invalidate? The `space_revision` counter (bumped on every logical content change) supports lazy invalidation: cache entries include a revision number; lookup misses if the space has moved on.

This is coarse-grained (any change invalidates all entries for that space). Fine-grained invalidation (tracking which terms changed and which caches depended on them) is an open research area.

5.4 Data Integrity and Security

For distributed backends, integrity becomes load-bearing infrastructure:

Revision vectors: Causal ordering of space mutations. When multiple nodes can write, version vectors or Lamport timestamps establish which update happened "before" another. Without this, "eventual consistency" is meaningless.

Immutable logs: Append-only logs of facts provide provenance and enable replay. DAS uses this pattern; so do blockchain-inspired knowledge graphs.

Backend parity testing: Same logical contents must produce identical results across backends. This is not just a test suite but a formal obligation—the `BackendSemanticParity` theorem.

Access control: Who can read or write which spaces? For AGI systems with self-modification, this is not a nice-to-have but a safety requirement.

This is not "security theater." A distributed knowledge base without integrity guarantees is a distributed bug generator.

6 Formal Backing

Key invariants should be mechanically verified, not just believed. This section surveys what formal obligations exist, where Lean formalization fits, and how confidence layers build upon each other.

6.1 The Layered Confidence Model

Verification is not all-or-nothing. We envision three layers:

Layer 1: Executable conformance. Cross-implementation test suites (e.g., `logicmoo/metta-testsuite`), differential testing against HE/PeTTa, property-based tests for evaluator equivalence, randomized backend parity tests. This is the foundation—without passing tests, nothing else matters.

Layer 2: Mechanized theorems. Lean proofs of load-bearing invariants: observable equivalence, TCO preservation, binding safety, hash-cons invariants, cache coherence, backend parity. These are confidence anchors: when a theorem is proved, we know with certainty that the property holds. When a theorem has `sorry`, we know exactly where our confidence stops.

Layer 3: Proof-producing reasoning. Selected external reasoners (ATP systems, specialized engines) return proof objects that CeTTa can consume or reconstruct. MeTTaMath and Lean integration provide bridges from reasoning to verification. This is the far horizon—genuinely exciting for AGI self-auditing scenarios.

More broadly, **derivation tracking** is an open question: can a user ask "how was this result derived?" Even E Prover has `-p` mode for proof output. Options range from simple unification traces to full proof terms. Whether this is optional (enabled by flag) or always available, and how much overhead it costs, remain design questions worth exploring.

6.2 Observable Equivalence

Two evaluation paths are equivalent if they produce the same result multiset (order may differ). This is the fundamental correctness criterion: if we have both recursive and frame-based evaluators, they must be observably equivalent.

6.3 Evaluator Path Theorems

`DirectVsStackEquivalence`: The recursive evaluator and the frame-based evaluator produce identical results on their shared domain.

`TCOPreservation`: The `TAIL_REENTER` path does not change semantics, only stack usage.

6.4 Binding and Substitution Safety

`VisibleBindingProjection`: Projecting only body-visible bindings does not change the outcome.

`BranchDeltaMerge`: Merging branch-local binding deltas back into the outer environment is sound.

`TypeDirectedArgumentOrder`: Evaluating arguments in type-directed order (strict first, lazy deferred) matches HE semantics.

6.5 Term and Storage Invariants

`HashConsInvariant`: Structurally equal terms share identity.

`ArenaResetSafety`: Published terms survive eval-arena rewind.

`SpaceRevisionCoherence`: Revision bumps and cache keys agree with logical mutation boundaries.

`BackendSemanticParity`: Same logical space contents produce identical results across backends.

6.6 Tabling and Cache Theorems

`SoundCaching`: Cached answers are correct answers (no false positives).

`CompleteAnswers`: When a table is marked complete, no additional answers exist.

`IncrementalInvalidationSoundness`: Dependency-based invalidation removes all affected entries.

Part III
The Journey

7 The Path We've Walked

This section describes what CeTTa has done, written as if we envisioned it from the start. The narrative is continuous because the work is continuous; we didn't know everything at the beginning, but each step followed from the previous.

7.1 Starting with HE Fidelity

The earliest decision: CeTTa would be a *direct* HE *runtime*, shaped around the HE evaluation structure. Not an interpreter for a different language that happens to parse MeTTa. Not a re-implementation that diverges where convenient. The HE spec is the contract.

This meant structuring `eval.c` around the mutual recursion of `metta`, `interpret_expression`, `interpret_function`, and friends. It meant threading expected type throughout, even when that was awkward. It meant implementing type-directed argument evaluation even though a simpler eager strategy would have been faster in some cases.

7.2 Arena Allocation and SymbolId

The first representation work: arena allocation for evaluation-local terms, avoiding malloc/free overhead. Then the `SymbolId` cutover, replacing string comparisons with integer comparisons. 38% instruction reduction. These were low-hanging fruit, but they set the pattern: representation changes pay off more than local optimizations.

7.3 SearchContext and ChoicePoint

As nondeterminism became more important, we introduced `SearchContext` and `ChoicePoint` as explicit structures. Small, real abstractions—not full SLG, but the beginning of it. `TableStore` followed, providing revision-aware memoization.

7.4 TAIL_REENTER and TCO

The `TAIL_REENTER` pattern emerged from necessity: deeply recursive programs were blowing the stack. A `goto tail_call` trampoline for deterministic single-result forms solved it. Now 15 call sites in `eval.c` use this pattern.

7.5 Where We Stand

The current position on the map:

- **Term representation**: Arena allocation, `SymbolId`, persistent hashcons—all working. Eval-local hashcons not attempted (needs arena-reset-safety confidence first).
- **Evaluation**: Type-directed, `TAIL_REENTER` for TCO, 15+ sites. Frame-based evaluation exists for streaming (`NativeResumeStack`), but typed-apply still recursive.
- **Search**: `SearchContext`/`ChoicePoint` as seams. `TableStore` for revision-aware memoization. Not full SLG.
- **Storage**: Native and MORK backends via `SpaceMatchBackend`. MORK read-side works; write-side deferred.
- **Formal**: Some Lean infrastructure in mettapedia. Key theorems have `sorry`s or are not started.

This is a mark on the map, not a redrawing of the map.

Part IV
The Horizon

8 Clouds of Possibility

What might we explore next? These are not stages to complete but regions where the landscape is promising and uncertain.

8.1 Eval-Local Term Sharing

Borrowing canonical identity into evaluation arenas could significantly reduce memory pressure on proof-heavy workloads. Positive witnesses include: recursive PLN chaining over medium knowledge bases, metamath proof search at various depths, pattern-heavy stdlib operations, and cross-backend parity tests. No single benchmark defines success; the workloads reveal which optimizations matter.

But it could also introduce lifetime complexity that costs more than it saves. The Conchon–Filliâtre model suggests a path, but the path is not walked.

8.2 Full SLG Tabling

XSB-style suspension/resumption/completion would handle recursive queries that currently loop or produce incomplete answers. The infrastructure is substantial. The question: which workloads need it? Recursive PLN chaining over large knowledge bases is the obvious candidate.

8.3 Dependent Types as Extension

MeTTa's type system could grow toward dependent types, where types can depend on values. This would enable richer verification and more precise staging. But it also complicates the "is this a value?" question significantly. A cloud to explore, not a near-term commitment.

8.4 `-lang petta` Mode

PeTTa diverges from HE in specific ways. A `-lang petta` flag could enable those divergences intentionally, allowing CeTTa to run PeTTa benchmarks without claiming they're HE-compliant. Useful for performance comparison and migration.

8.5 PLN/ATP as Optional Lanes

Backward chaining, forward chaining, ATP integration (Lash-style or otherwise)—these are specialized inference modes. They should integrate through explicit lane interfaces, not as modifications to the core evaluator. The `mork:` lane prefix is a start.

8.6 Rho-Calculus Lane

Meredith's reflective higher-order process calculus [20] provides formal process-algebraic semantics for concurrent computation with reflection. MeTTa's quote/unquote semantics map naturally to rho-calculus's `@` and `*` operators, suggesting a principled foundation for concurrent MeTTa execution.

This could enable:

- **Decentralized execution**: RChain-style distributed computation where processes run on different nodes and communicate via channels.
- **Formal concurrency semantics**: No ad-hoc parallelism. The process algebra defines what "concurrent evaluation" means.
- **Reflection as first-class**: In rho-calculus, code-as-data is native, not an encoding. This aligns with MeTTa's reflective design.

Open question: how much rho-calculus machinery does CeTTa need versus borrow? Full Rholang compilation? A lightweight embedding? A translation layer that preserves key properties?

9 The Cloud of Forks

The endgame is not a single design point. It is a region of design space with multiple viable paths. Here we enumerate the major decision axes where multiple options remain open.

9.1 Representation Fork

Option A: Persistent-only hash-consing. Simplest ownership story. Fastest to stabilize. But leaves evaluation-time copying costs.

Option B: Arena-disciplined borrowed sharing. Stronger memory savings. Aligns with one-universe ideal. Needs airtight publication/lifetime rules and formal proof.

Option C: Mostly canonical + local wrappers. Deepest sharing. Strongest long-term symbolic performance. Hardest implementation and proof burden.

9.2 Substitution Fork

Option A: Eager substitution. Simple mental model. Can be expensive on deep substitution chains.

Option B: Selective delayed substitution. Middle ground, focusing on hot cases only.

Option C: Explicit substitutions everywhere. Radically improved copying behavior. Evaluator and normal-form logic become more complex (Abadi et al. $\lambda\sigma$-calculus).

9.3 Evaluator/Machine Fork

Option A: Direct recursive/trampoline. Clearest semantics, easiest to debug. May suffice for deterministic paths.

Option B: Explicit resumable frame machine. Supports streaming and bounded consumers. Risk of destroying tail-path performance if not careful.

Option C: Derived/compiled machine family. Best performance ceiling. Greatest need for observable-equivalence proofs.

9.4 Tabling Fork

Option A: Revision-aware cache only. Lowest complexity. Insufficient for serious recursion/-cycles.

Option B: Variant tabling. Strong step up, simpler than subsumption.

Option C: Subsumptive tabling. More reuse, more subtle semantics/performance tradeoffs.

Option D: Incremental tabling. Strongest fit for changing knowledge bases. Higher complexity.

9.5 Ordering/Demand Fork

Option A: Strict ordered eager results. Simplest user story. Can overcompute badly.

Option B: Ordered but bounded-demand streaming. Likely best near-term target. Pengines-style on-demand.

Option C: Relaxed-order parallel modes. Strongest scalability. Must be explicit semantically.

9.6 Backend/Storage Fork

Option A: Native array/set backend as baseline. Simple, clear benchmark baseline.

Option B: PathMap/trie-heavy backend. Good algebraic/path operations. Risk of conflating membership with identity.

Option C: MORK/graph DB backend. High ceiling for graph-native workloads. Requires strong parity discipline.

Option D: Distributed (DAS) backend. Very large knowledge bases across machines. Explicit cost/consistency model.

Option E: Hybrid multi-backend. Likely true endgame. Requires clean interfaces and parity tests.

9.7 Verification Fork

Option A: Tests only. Insufficient long-term confidence.

Option B: Tests + selected Lean theorems. Practical and high value. The near-term sweet spot.

Option C: Proof-producing kernel / checked delegations. Strongest long-term trust story. More ambitious but strategically valuable.

A Research Annex

Ideas that inspire creativity but aren't on the main map.

Factor-Graph Demand/Supply. Ben's factor-graph approach to PLN inference, where demand flows backward and supply flows forward through a graph of inference rules. Promising for large-scale reasoning, but not the evaluator core.

ECAN-Style Attention. Economic attention allocation for focusing computational resources on important atoms. Interesting for AGI applications, orthogonal to the runtime's core concerns.

E-Graphs and Equality Saturation. The `egg` and `egglog` projects show how congruence closure, equality saturation, and Datalog can combine for powerful rewrite optimization. A potential lane, not the evaluator core.

Compiled Datalog (Soufflé). For monotone or stratified logical fragments, Soufflé-style compilation into parallel C++ can yield dramatic speedups. Recognizing when a subproblem has Datalog shape and compiling it aggressively is a long-term optimization opportunity.

OSLF and Type Hypercubes. The OSLF framework [21] shows how to extract a family of type systems (organized as a hypercube) from operational rewrite rules. The modal layer with $\langle K_j \rangle$ modalities and equational center $\mathcal{Z}$ can help classify which operations are pure vs. effectful and which rewrites commute (enabling safe parallelism). MeTTa's spatial-behavioral type annotations could interface with this framework, deriving effect structure from the rules themselves rather than imposing it externally.

Worst-Case Optimal Joins. For large conjunctive matching, Leapfrog Triejoin [22] and related worst-case optimal join algorithms matter. Variable ordering and multiway join planning become important when MeTTa queries involve multiple patterns over large spaces.

Historical Notes. A git-generated timeline of major commits and version tags would go here. The main roadmap describes territory, not chronology.

B Source Corpus

This section catalogs the primary sources for CeTTa implementation, organized by role. Each entry explains *why it matters* for the endgame.

B.1 Hyperon / MeTTa Semantics

`trueagi-io/hyperon-experimental` The main public MeTTa implementation. Contains the HE spec in `docs/metta.md`. The semantic reference against which CeTTa must be compared.

Meta-MeTTa (arXiv 2305.17218) Operational semantics for MeTTa. Provides a formal foundation for what evaluation *means*.

Reflective Metagraph Rewriting (arXiv 2112.08272) Conceptual framing for MeTTa as reflective metagraph rewriting. Important for understanding the broader Hyperon vision.

B.2 Comparative Implementations

`trueagi-io/PeTTa` Efficient Prolog/SWI-based MeTTa. Performance comparator and interoperability target. Shows what WAM-style efficiency looks like.

`trueagi-io/metta-wam` WAM-targeting MeTTa implementation. Useful for understanding compiled control flow.

`logicmoo/metta-testsuite` Cross-implementation compatibility harness. Essential for conformance testing.

`singnet/das` Distributed Atomspace. Large-scale backend with MeTTa-integrated query API. Shows how CeTTa could scale to million-atom KBs.

B.3 Search/Control/Tabling

Warren 1983 The WAM paper. Classic machine model for efficient Prolog. Choicepoints, trail, last-call optimization.

XSB Manual SLG-WAM, tries, call subsumption, incremental maintenance, shared tables. The reference for serious tabling implementation.

SWI-Prolog Tabling Docs Modern production view: JIT indexing, delimited continuations, incremental tabling.

Desouter et al. 2016 "Tabling as a Library with Delimited Control." Shows how tabling can be implemented cleanly with delimited continuations.

B.4 Term Representation / Indexing

Conchon & Filliâtre 2006 Type-safe modular hash-consing. Canonical reference for safe structural sharing. Weak references, GC integration, lifetime safety.

Graf 1994 Substitution tree indexing. Produces bindings during traversal— exactly what MeTTa pattern matching needs.

Schulz (E Prover) One shared term bank plus multiple specialized indices. The converged architecture after years of "mess."

Zhu et al. 2025 Modern hash-consing in JuliaSymbolics. 3.2× speedup, 2× memory reduction on real biological models. Validates the payoff.

B.5 Evaluator/Machine Design

Ager et al. (Danvy) "A Functional Correspondence between Evaluators and Abstract Machines." Derive machine structure from evaluator semantics systematically.

Maurer et al. ("Compiling without continuations") Join points as first-class optimization concept. Preserves non-escaping tail paths as direct jumps.

Leijen & Lorenzen ("Tail Recursion Modulo Context") Advanced tail-call optimization that supports structured hot paths without losing abstraction.

B.6 Optional Reasoning Lanes

egraphs-good/egg High-performance equality saturation. E-graph lane for congruence closure and rewrite optimization.

egglog Unifies Datalog and equality saturation. Incremental execution, cooperating analyses, lattice reasoning.

souffle-lang/souffle Highly optimized Datalog with parallel native compilation. Shows how to compile logical fragments aggressively.

Differential Dataflow "Incremental updates with millisecond propagation." Inspiration for incremental knowledge maintenance.

trueagi-io/hyperon-pln MeTTa-native PLN. The uncertain reasoning lane.

B.7 Verification / Proof

Lean 4 + mathlib Target environment for mechanized theorems. Metaprogramming book for tactic development.

MeTTaMath Bridge between MeTTa and verifiable proof kernels. Metamath-style checking.

vprover/vamplean Proof-producing ATP import into Lean. Model for trusted external reasoning integration.

E Prover, Vampire Mature ATP systems. Potential coprocessors for delegated theorem proving.

TPTP Benchmark/format/evaluation backbone for ATP work. Common problems and standards.

References

[1] S. Conchon and J.-C. Filliâtre. Type-safe modular hash-consing. In *ML Workshop*, 2006. https://usr.lmf.cnrs.fr/~jcf/publis/hash-consing2.pdf

[2] P. Graf. Substitution tree indexing. In *RTA*, 1994. https://pure.mpg.de/rest/items/item_1834191_2/component/file_1857890/content

[3] D.H.D. Warren. An abstract Prolog instruction set. Technical Note 309, SRI International, 1983. https://www.sri.com/wp-content/uploads/2021/12/641.pdf

[4] The XSB System, Version 4.0, 2022. https://xsb.sourceforge.net/

[5] S. Schulz. E—a brainiac theorem prover (Version 3.x, 2024). *AI Communications*, 2002 (original). https://github.com/eprover/eprover

[6] B. Desouter, M. van Dooren, and T. Schrijvers. Tabling as a library with delimited control. *IJCAI*, 2016. https://www.ijcai.org/Proceedings/16/Papers/619.pdf

[7] TrueAGI. Hyperon Experimental. https://github.com/trueagi-io/hyperon-experimental

[8] TrueAGI. PeTTa: Prolog-based MeTTa. https://github.com/trueagi-io/PeTTa

[9] SingularityNET. Distributed Atomspace (DAS). https://github.com/singnet/das

[10] A. Vandervorst. PathMap: Prefix-compressed structural sharing. https://github.com/Adam-Vandervorst/PathMap

[11] TrueAGI. MORK: High-performance hypergraph kernel. https://github.com/trueagi-io/MORK

[12] egg: E-graphs good. https://github.com/egraphs-good/egg

[13] Soufflé: High-performance Datalog. https://souffle-lang.github.io/

[14] Lean 4 theorem prover. https://github.com/leanprover/lean4

[15] B. C. Smith. Reflection and semantics in Lisp. In *POPL*, 1984.

[16] N. Amin and T. Rompf. Collapsing towers of interpreters. In *POPL*, 2018. https://dl.acm.org/doi/10.1145/3158140

[17] U. Berger and H. Schwichtenberg. An inverse of the evaluation functional for typed λ-calculus. In *LICS*, 1991.

[18] R. E. Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1):97–109, 1985.

[19] D. P. Friedman, W. E. Byrd, and O. Kiselyov. *The Reasoned Schemer*. MIT Press, 2005.

[20] L. G. Meredith and M. Radestock. A reflective higher-order calculus. In *ENTCS*, 2005.

[21] M. Stay, L. G. Meredith, and C. Wells. Generating Hypercubes of Type Systems. 2026.

[22] T. L. Veldhuizen. Leapfrog Triejoin: A Simple, Worst-Case Optimal Join Algorithm. In *ICDT*, 2014. https://openproceedings.org/2014/conf/icdt/Veldhuizen14.pdf

[23] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Lévy. Explicit substitutions. *Journal of Functional Programming*, 1(4):375–416, 1991.

[24] SWI-Prolog Pengines: Web Logic Programming Made Easy. https://www.swi-prolog.org/pldoc/doc_for?object=section(%27packages/pengines.html%27)